NumCompute-Stream: A Buffered Decision Tree Streaming Framework

Edward Coombs · a984678 · COMP 5004 Programming for AI · 16 November 2026

1. Overview

NumCompute-Stream extends our group's 2.1 NumCompute package into a streaming learner built around one decision tree. Data arrives in chunks, and every component updates from each chunk with bounded memory, using only NumPy and matplotlib. The demo streams a 12 000-row sensor CSV whose labelling rule changes halfway through.

There are 99 new test functions (757 tests with the 2.1 baseline), all passing with warnings treated as errors, and flake8 is clean. The vectorised split search matches a loop version of the course tree exactly and runs 97–167× faster.

2. Design Decisions

2.1 Structure

`io.load_csv` reads the file, `io.iter_chunks` splits it into chunks, and `StreamTrainer` takes them one at a time. It scores each chunk before training on it (test-then-train) and logs metrics and memory use per chunk. Training goes through `Pipeline.partial_fit`, which updates each step in turn.

Every transformer and estimator inherits a `partial_fit` from its base class that raises `NotImplementedError`, so `hasattr` cannot tell which classes really stream. Those that do set a `supports_partial_fit = True` flag. `Pipeline` checks this flag on every step, and checks that X and y have the same length, before it changes any state.

Each preprocessing class's `fit` resets the object and then calls `partial_fit`. Batch and streaming therefore share one code path and cannot drift apart, and the 2.1 contract tests pass unchanged.

Every chunk is validated in full before any state changes. A rejected chunk leaves a metric, encoder or tree as it was, so a long stream can catch the error and carry on. Infinity is always rejected, since one `inf` would turn a running variance into NaN for the rest of the stream.

Existing code was reused where it fitted. The `Imputer` is built on `StreamingStats` and `StreamingQuantiles`, and the batch and streaming metrics share the 2.1 formulas.

Table 1 maps each requirement in the specification to the code that implements it.

Table 1. Specification requirements and where they are implemented.

Requirement	Implementation
Streaming mean, variance and quantiles; `update_stats`	`StreamingStats`, `StreamingQuantiles`
Histograms with a sliding window	`StreamingHistogram(bins, value_range, window)`
Scalers with running statistics	`StandardScaler`, `MinMaxScaler`
Imputer updated on the fly	`Imputer` (mean, median or constant)
One-hot encoding that grows	`OneHotEncoder`; new categories become new columns at the end
Metrics with `update`, `reset`, `result`	`Accuracy`, `Precision`, `Recall`, `F1`
Confusion matrix and AUC over time	`StreamingConfusionMatrix`, `StreamingAUC`
Rolling-window metrics	`window=w` on every streaming metric (last w chunks)
Buffered decision tree	`DecisionTreeClassifier` (`max_depth`, `min_samples_split`, `max_features`, `buffer_size`, Gini or entropy)
Pipeline with `partial_fit`	`Pipeline.partial_fit`, `predict`, `predict_proba`
Stream trainer with logging	`StreamTrainer.fit_chunk`, `score_chunk`; `memory_footprint`
Three plots, saved or shown	`plot_metric_over_time`, `compare_models`, `plot_predictions_vs_ground_truth`
CSV loading and chunking	`io.load_csv`, `io.iter_chunks`

2.2 Algorithms

Where the right algorithm was not obvious, I ran a short spike on example data first.

Mean and variance. `StreamingStats` uses Welford's algorithm in its batch form, vectorised across columns with per-column NaN masks. Near 1e9 its relative error is about 1e-8, the float64 floor there. The textbook $E[x^2] - E[x]^2$ is out by a factor of 130. Storing the mean and M2 in power-of-two units keeps the 2.1 ±1e308 tests passing.

Quantiles vs AUC. Both spikes compared a histogram with a random sample, and they reached opposite answers.

Table 2. Bounded-memory summaries chosen for quantiles and AUC, based on the two spikes.

Use	Choice	Reason
Quantiles (median `Imputer`)	Reservoir sample, k = 1000	The histogram fails on heavy tails: on Cauchy data the adaptive histogram's median rank error reached 0.499. The reservoir stays near 0.015 on every distribution tested.
AUC	Histogram, 1000 bins over [0, 1]	The range is known. Exact when distinct scores fall in distinct bins (usual for leaf probabilities), 10–1000× more accurate than a reservoir, and `max_error()` gives a bound.

Split search. The course tree tests one mask per question, which is $O(n^2 d)$ per node once features are numeric. At each node, mine sorts every feature once, one-hot encodes the labels in sorted order and takes a `cumsum`. That gives the left-hand class counts for every threshold at once, and the gain for all candidates in one array.

The result is the same as the course algorithm. Its tie-breaks still hold (lowest feature, then lowest threshold; a majority tie goes to the smallest class), and on 800 random cases (400 problems, each with Gini and entropy) it chose the same splits as a loop reference, with gains equal to the last bit. Thresholds are computed as `a/2 + b/2`, which cannot overflow for values near 1e308.

The tree is stored as flat arrays and built with an explicit stack, so a depth of 2999 runs without hitting Python's recursion limit. `predict` moves every row down one level per step, so its Python loop runs at most `depth` times whatever the number of rows. Class counts per node give `predict_proba`, which AUC needs.

Buffering. `partial_fit` copies each chunk into the buffer, retrains once it holds `buffer_size` rows, then clears it. `classes_` holds every label seen so far, so the probability columns stay stable. Before the first retrain `predict` raises, and `StreamTrainer` logs NaN for those chunks.

3. Testing and edge cases

3.1 Edge cases

A value that is undefined partway through a stream is returned as NaN, which shows as a gap in the plot. A mistake by the caller raises an error.

Table 3. Streaming edge cases and how the library handles them.

Edge Case	Behaviour
AUC, precision or recall with one class seen	NaN with no 0/0 warning (warnings fail the tests)
String labels with the default `pos_label=1`	`ValueError`; NaN forever would hide the bug
Constant column	Variance exactly 0, so scalers use scale 1
All-NaN column in an early chunk	Allowed by `partial_fit`, rejected by `fit`; the `Imputer` uses `fill_value`
NaN reaching the tree	Rejected; the Imputer has to run first
±1e308 in different chunks, subnormal neighbours	2.1 half-range trick; midpoint fallback
New category mid-stream	New column at the end, so existing columns keep their meaning
Strings and numbers mixed across chunks	`TypeError` from the metrics and `OneHotEncoder`
Rolling window larger than the data seen	Uses every chunk seen so far
−0.0 and 0.0 in one feature	Treated as equal, so the tree does not split them
Single-class buffer mid-stream	The tree becomes one leaf; `classes_` keeps earlier labels at probability 0

3.2 Tests and debugging

The 99 new test functions split as: stats 32, preprocessing 18, validation 10, metrics 9, tree 9, io 7, pipeline 5, stream 4, visualise 3, integration 2. Most modules were written test-first; the tree split search, quantiles and AUC were spiked first. Where I could, tests compare against an independent reference: a loop tree inside the test file, or NumPy on the full array. Plots are tested on their data and checked by eye, and the eye check caught a legend covering the points that the tests missed.

The integration tests run a CSV file with blank cells and text labels through the pipeline and `StreamTrainer`, then save the plots. They passed first time, so I broke the code on purpose to check that they could fail. With the wrong AUC column one test failed, and with a buffer that never clears both failed. The second test streams a shuffled, seeded CSV twice and requires identical logs. It also checks that peak model memory after 4500 rows is within 10% of the peak after 1500 rows, which shows memory is set by `buffer_size`.

Bugs found during testing included:

- The tree buffer kept references to the caller's arrays, so a caller that reused them changed the buffered data. I wrote a failing test, then made the buffer copy each chunk.
- With string labels and the default `pos_label`, AUC returned NaN forever. I found this by probing after the tests passed, and it now raises a `ValueError`.

- A chunk with mixed types in a later feature left new categories in the earlier `OneHotEncoder` features, and a wrong y length was only caught by the last pipeline step. Both now check the whole input before changing state.
- A variance test near 1e9 failed at `rtol=1e-9`. After comparing three formulas I found the tolerance was below what float64 can resolve at that size, so I set it to 1e-6 and added an assertion that the naive formula fails.
- Moving `StandardScaler` onto `StreamingStats` broke four 2.1 tests that use extreme magnitudes (1e-200 to ±1e308). I fixed the cause in `StreamingStats` by storing values in power-of-two units, and the four tests pass unchanged.
- One bug was in a test. A tie-breaking test reused `y` after it had been reassigned, so its node was pure and a single leaf was the correct answer. I fixed the test.

4. Benchmark results

Timings come from `benchmark/run_all.py` (NumPy 2.5.4, GitHub codespace). Each is the minimum over interleaved runs after a warm-up, and each pair was checked to return the same result first.

4.1 Loop vs vectorised

Table 4. Loop-based and vectorised run times. Each value is the minimum over interleaved runs, and each pair returns identical results.

Operation	Loop (ms)	Vectorised (ms)	Speed-up
Split search, n = 250, d = 8, k = 3	49.1	0.51	97×
Split search, n = 1000	246.6	2.27	109×
Split search, n = 4000	1769.1	10.59	167×
Tree predict, 20 000 rows, depth 8	102.0	3.47	29×
Accuracy + macro F1 + AUC, 50 chunks × 200	4.1	11.59	0.4×
Same, 50 chunks × 2000	39.0	22.24	1.8×
Same, 50 chunks × 20 000	396.2	123.76	3.2×

The split-search gap grows with n, because the loop is $O(n^2 d)$ and the sorted version $O(nd \log n)$. The streaming metrics are slower on small chunks because each one validates its chunk, a fixed cost that my bare dictionary loop does not suffer. I kept the checks, since at realistic chunk sizes they cost little. Memory use stays bounded. In checks during development, passing 500 000 samples through the three metrics retained about 20 kB, and in one check the logged model memory was 17 600 B against 17 840 B from `tracemalloc`. The gap is Python object headers, and the measurement costs 0.04 ms per call, so it is logged on every chunk.

4.2 Effect of buffer_size

The stream has 24 000 rows in chunks of 100, with 5% label noise and 3% missing values, and the labelling rule changes at chunk 120. The pipeline is an Imputer, a StandardScaler and a tree (`max_depth=6`), scored test-then-train over three seeds. I report rolling accuracy because cumulative accuracy hides the drift.

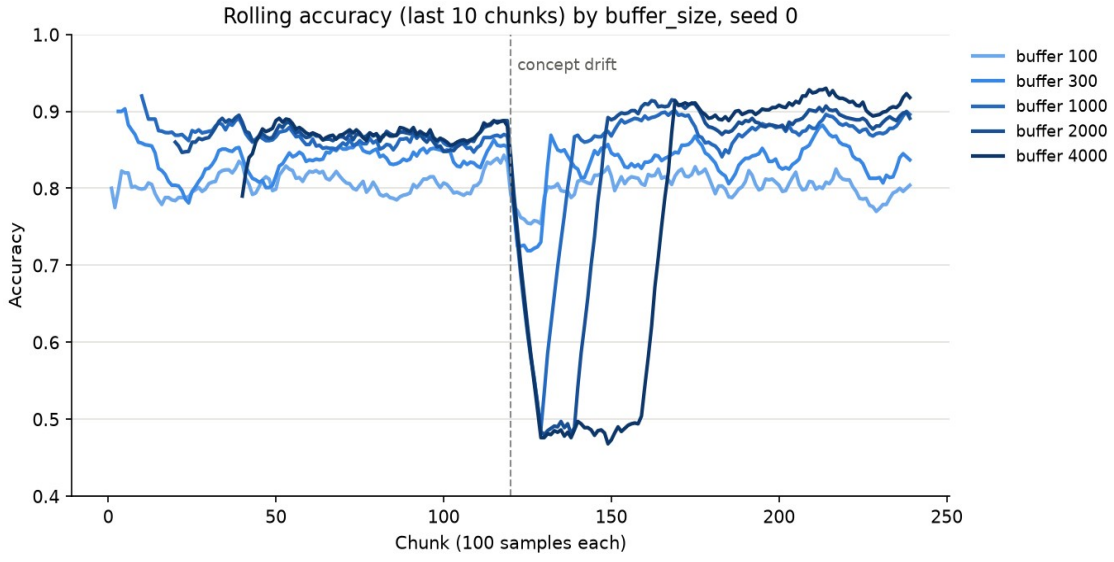


Figure 1. Rolling accuracy (last 10 chunks) for each buffer_size, seed 0. Produced by `benchmark/bench_buffer_size.py`.

A bigger buffer is more accurate on stable data, but the tree only learns the new rule once a full buffer of post-drift rows has arrived. Recovery takes 5 chunks with a 100-row buffer and 45 with a 4000-row buffer. Here the drift falls on a buffer boundary for every size. In general a buffer that is part-full at the drift first trains one tree on mixed data, so recovery can take up to 2 × buffer_size rows. Over the drifted part of the stream a medium buffer (1000) did best. Each retrain gets slower as the buffer grows, from 2.2 ms to 26.4 ms, but total fit time falls from 0.54 s to 0.22 s because there are 40 times fewer retrains. Memory grows linearly, from 3 kB to 174 kB.

The demo shows the same trade-off on its 12 000-row sensor stream (Figure 2). With a 300-row buffer, rolling accuracy dips to about 0.71 at the drift and recovers within about ten chunks. With a 2000-row buffer it falls to about 0.53 and takes about 30 chunks, but it then settles near 0.90 compared to about 0.84 for the 300 row.

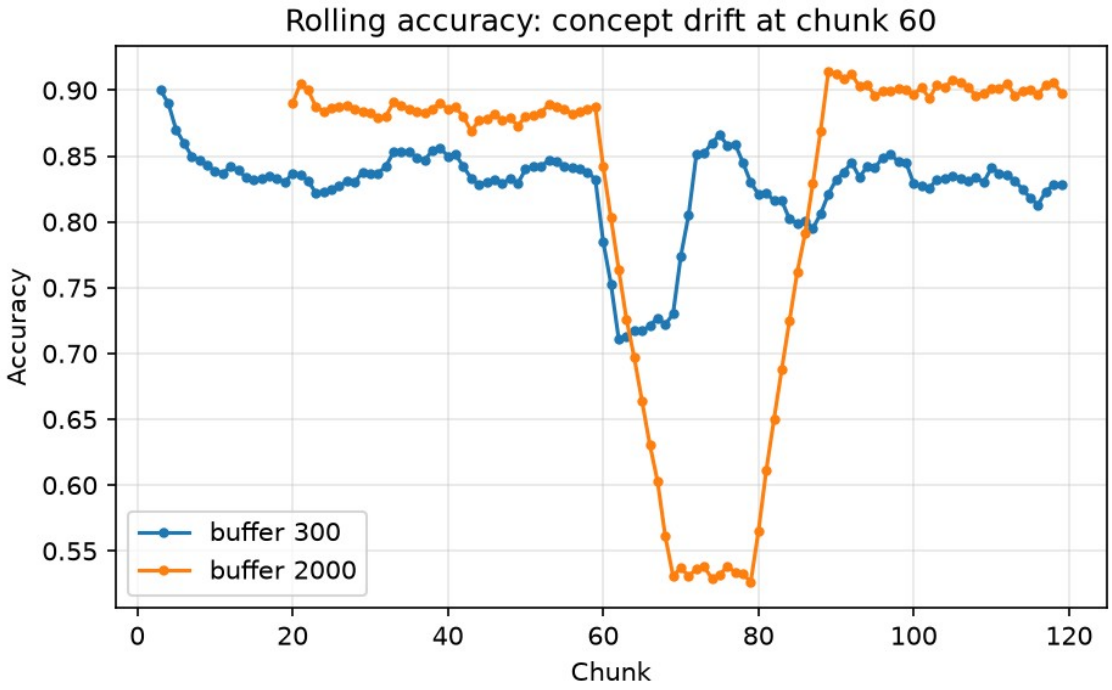


Figure 2. Rolling accuracy (last 10 chunks) on the demo sensor stream, buffer 300 against 2000, with the drift at chunk 60. Drawn with `compare_models` *in* `demo/stream_demo.ipynb`.

5. Reflections

Running spikes before committing to an algorithm was worth the time. I expected histograms to win for quantiles, and they failed on heavy tails. For AUC they did win, because the range of a probability is known in advance. I also learned that a numerically stable method such as Welford keeps the error close to the float64 floor, and a test has to allow for that floor. Treating warnings as errors was a nuisance at first, but it meant every 0/0 case had to be handled on purpose.

The buffer sweep showed the cost of the spec's design. Retraining on the latest buffer forgets older data, which helps under drift and hurts on stable data. A drift detector that triggers an early retrain could give the accuracy of a large buffer with the recovery speed of a small one, and it is the first thing I would add given more time.

Known limitations:

- `Pipeline` has no column selection, so categorical columns must be encoded before it. A new category mid-stream widens the `OneHotEncoder` output and the pipeline raises; fitting the encoder on all categories first avoids it.
- If a later step raises on that width check, earlier steps have already updated.
- Buffered rows were scaled with the scaler's statistics at arrival time.
- The split search holds n · d · k int64 counts, about 20 MB at 5000 × 50 × 10.